

PE6201 · Class 2 — RAG, the token-free version

A tiny **retrieval-augmented generation (RAG)** pipeline you can run end-to-end **for free** — no API key, no tokens. Everything runs locally with open-source models.

How to use: run the cells top to bottom. You don't have to write any code — look for the lines marked **TRY THIS** and change them.

RAG in one line: **embed your notes -> retrieve the ones nearest your question -> let a model answer from them.**

Setup

Installs the open-source libraries (this also brings in PyTorch). On Google Colab this takes a minute the first time.

```
In [100... %pip -q install sentence-transformers transformers
```

Note: you may need to restart the kernel to use updated packages.

The knowledge base

Eight short notes from Class 2. This is what the model is allowed to answer from.

```
In [101... # Our tiny knowledge base = 8 short notes from Class 2.
# In a real system this would be thousands of documents; the pipeline is identical.
DOCS = [
    "The modern AI stack has layers: compute, data and embeddings, a vector store, the model, orchestration, serving",
    "Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks",
    "Build-vs-buy is a decision per layer. Firms usually own their data and governance, rent frontier models, and manage",
    "Training a model is a one-off cost; inference is paid on every request and usually dominates at scale. An H100",
    "Evaluation is a layer of the stack, not an afterthought. AI can fail silently: a fluent, confident answer that",
    "DBS, Singapore's largest bank, built its own data platform (ADA) and a responsible-data framework called PURE",
    "Fine-tuning bakes knowledge and behaviour into a model's weights and is good for a stable style or format. Retrieval",
    "Embeddings turn text into vectors so that similar meanings land near each other; retrieval finds the vectors nearest",
]
print(f"{len(DOCS)} documents loaded")
```

8 documents loaded

Chunking — why we split (and overlap)

Before embedding, long documents are cut into **chunks**. Why not just embed a whole document?

- **Retrieval precision** — retrieval should return the relevant *passage*, not a 40-page manual. One vector per chunk means nearest-neighbour search finds the specific paragraph that answers the question.
- **One embedding = one meaning** — an embedding is a single point in meaning-space. Embed a long, multi-topic doc into one vector and you get a blurry *average* that matches everything a little and nothing well. Small chunks keep each vector about a single idea, so cosine similarity stays sharp.
- **Context window + cost** — the retrieved text is pasted into the prompt. Whole documents blow past the model's token limit, and every token costs money and latency — so we fetch a few small, on-target chunks.
- **Embedding input limits** — embedding models truncate long inputs anyway, silently dropping the tail.

Why overlap? Cut on a hard boundary and an idea that straddles two chunks gets split — each half is incomplete and neither retrieves as a full answer. Overlapping neighbours by a few words keeps a boundary idea whole in at least one chunk.

It's a **knob you tune against your evals**: too big → diluted, noisy vectors and wasted tokens; too small → each chunk loses the context needed to make sense. Turn `CHUNK_WORDS` / `OVERLAP` below and watch the split change.

```
In [102... # TRY THIS (knob): chunk size (in words) and overlap between neighbours.
CHUNK_WORDS = 25      # try 12 (tiny chunks) or 200 (whole note = 1 chunk)
OVERLAP      = 10      # words shared with the next chunk, so an idea isn't cut in half

def chunk(text, size=CHUNK_WORDS, overlap=OVERLAP):
    words = text.split()
    if len(words) <= size:
        return [text]
    step = max(1, size - overlap)
    out = []
    for i in range(0, len(words), step):
        out.append(' '.join(words[i:i+size]))
        if i + size >= len(words):      # this chunk already reached the end
            break
    return out
```

```

# Build the chunks the retriever will use, from the knob above.
CHUNKS = [c for d in DOCS for c in chunk(d)]
print(f'{len(DOCS)} docs -> {len(CHUNKS)} chunks (CHUNK_WORDS={CHUNK_WORDS}, OVERLAP={OVERLAP})')

# See how each document is split, and the words neighbouring chunks SHARE (the overlap).
for d_i, doc in enumerate(DOCS, 1):
    cs = chunk(doc)
    print(f'\nDoc {d_i}: split into {len(cs)} chunk(s)')
    for c_i, c in enumerate(cs):
        print(f'    chunk {c_i} ({len(c.split())}w): {c}')
    if len(cs) > 1:
        shared = ' '.join(cs[0].split()[-OVERLAP:])
        print(f'    -> chunks 0 & 1 overlap by {OVERLAP} words: "{shared}"')

```

8 docs -> 17 chunks (CHUNK_WORDS=25, OVERLAP=10)

Doc 1: split into 2 chunk(s)

chunk 0 (25w): The modern AI stack has layers: compute, data and embeddings, a vector store, the model, orchestration, serving, and observability/evals. Each layer has a job, a

chunk 1 (15w): orchestration, serving, and observability/evals. Each layer has a job, a failure mode, and a cost.

-> chunks 0 & 1 overlap by 10 words: "orchestration, serving, and observability/evals. Each layer has a job, a"

Doc 2: split into 2 chunk(s)

chunk 0 (25w): Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks most similar to a question, and generate an answer from those

chunk 1 (19w): similar to a question, and generate an answer from those chunks. The model is rented; the retrieval is yours.

-> chunks 0 & 1 overlap by 10 words: "similar to a question, and generate an answer from those"

Doc 3: split into 2 chunk(s)

chunk 0 (25w): Build-vs-buy is a decision per layer. Firms usually own their data and governance, rent frontier models, and mix at the serving layer, judged on cost,

chunk 1 (16w): models, and mix at the serving layer, judged on cost, latency, control, data gravity, and regulation.

-> chunks 0 & 1 overlap by 10 words: "models, and mix at the serving layer, judged on cost,"

Doc 4: split into 3 chunk(s)

chunk 0 (25w): Training a model is a one-off cost; inference is paid on every request and usually dominates at scale. An H100 GPU rents for roughly 2

chunk 1 (25w): dominates at scale. An H100 GPU rents for roughly 2 to 7 US dollars per hour, and large models cost a few dollars per million

chunk 2 (11w): hour, and large models cost a few dollars per million tokens.

-> chunks 0 & 1 overlap by 10 words: "dominates at scale. An H100 GPU rents for roughly 2"

Doc 5: split into 2 chunk(s)

chunk 0 (25w): Evaluation is a layer of the stack, not an afterthought. AI can fail silently: a fluent, confident answer that is simply wrong. So you must

chunk 1 (21w): fluent, confident answer that is simply wrong. So you must measure quality with an eval set and monitor it in production.

-> chunks 0 & 1 overlap by 10 words: "fluent, confident answer that is simply wrong. So you must"

Doc 6: split into 2 chunk(s)

chunk 0 (25w): DBS, Singapore's largest bank, built its own data platform (ADA) and a responsible-data framework called PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation

chunk 1 (25w): PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation models. Its

DBS-GPT assistant pairs rented models with an internal knowledge base, which is RAG.

-> chunks 0 & 1 overlap by 10 words: "PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation"

Doc 7: split into 2 chunk(s)

chunk 0 (25w): Fine-tuning bakes knowledge and behaviour into a model's weights and is good for a stable style or format. Retrieval looks facts up at query time

chunk 1 (23w): style or format. Retrieval looks facts up at query time and is good for private or changing data.

Most teams start with retrieval.

-> chunks 0 & 1 overlap by 10 words: "style or format. Retrieval looks facts up at query time"

Doc 8: split into 2 chunk(s)

chunk 0 (25w): Embeddings turn text into vectors so that similar meanings land near each other; retrieval finds the vectors nearest the question. Chunking and the number of

chunk 1 (22w): the vectors nearest the question. Chunking and the number of retrieved chunks (top-k) usually decide RAG quality more than the model does.

-> chunks 0 & 1 overlap by 10 words: "the vectors nearest the question. Chunking and the number of"

Try it in another language — / हिन्दी

The default embedder is **English-only**, so Chinese / Hindi retrieval will be weak. To run this pipeline in another language: (1) paste one document set below into `DOCS` and re-run the corpus + chunking + embed cells, (2) pick a **multilingual embedder** in Stage 1, and (3) pick a **multilingual model** in Stage 3.

Chunking caveat: the toy `chunk()` splits on spaces — fine for Hindi, but **Chinese has no spaces**, so word-splitting fails. For Chinese, chunk by characters or by sentence (split on `。`), or use a segmenter such as `jieba`.

Chinese () — copy into `DOCS` :

```
DOCS = [
    "                RAG",
    "    ",
    "    ",
    "    ",
    "    DBS                ADA                AI                top-k                ",
    "    RAG    ",
    "    ",
    "    DBS-GPT",
    "    ",
    "    "
]
```

Hindi (हिन्दी) — copy into `DOCS` :

```
DOCS = [
```

"रिट्रीवल-ऑगमेंटेड जनरेशन (RAG) के तीन चरण हैं: दस्तावेज़ों को वेक्टर में बदलना, प्रश्न से मिलते-जुलते हिस्सों को खोजना, और उन्हीं हिस्सों से उत्तर तैयार करना। मॉडल किराये का है, रिट्रीवल आपका अपना।",

"एम्बेडिंग टेक्स्ट को वेक्टर में बदलती है ताकि समान अर्थ वाले वाक्य पास-पास रहें; चंकिंग और top-k अक्सर मॉडल से ज़्यादा गुणवत्ता तय करते हैं।",

"मूल्यांकन (evals) स्टैक की एक परत है, बाद की सोच नहीं: AI चुपचाप गलत हो सकता है, इसलिए गुणवत्ता मापना ज़रूरी है।",

```
]
```

Sample text for practice — swap in your own domain documents.

Stage 1 — Embed

Turn each chunk into a vector (a list of numbers) so that similar meanings sit near each other. We use a small open-source embedding model that runs on CPU.

```
In [103... from sentence_transformers import SentenceTransformer

# TRY THIS (knob): swap the embedding model — different models output different-size vectors.
# 'all-MiniLM-L6-v2'          -> 384 dims (tiny, fast; default; ENGLISH-only)
# 'all-mpnet-base-v2'         -> 768 dims (larger, higher quality; English)
# 'BAAI/bge-small-en-v1.5'    -> 384 dims (strong small model; English)
# multilingual ( / हिन्दी): 'intfloat/multilingual-e5-base' (768) · 'BAAI/bge-m3' (1024) · 'sentence-transformers
EMBED_MODEL_NAME = 'all-MiniLM-L6-v2'

embedder = SentenceTransformer(EMBED_MODEL_NAME) # downloads once

def embed(texts):
    # normalize so a dot product equals cosine similarity
    return embedder.encode(texts, normalize_embeddings=True)

doc_vecs = embed(CHUNKS)
print(f'{EMBED_MODEL_NAME}: embedded {doc_vecs.shape[0]} chunks into {doc_vecs.shape[1]}-dim vectors')
```

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 2990.10it/s]
```

```
all-MiniLM-L6-v2: embedded 17 chunks into 384-dim vectors
```

Embedding models differ — dimensionality, quality, speed

The number printed above (e.g. **384**) is the **dimensionality** — how many numbers describe each chunk's meaning. It's fixed per model, and every vector in your store must come from the *same* model.

- **More dimensions** $\approx$ more capacity for nuance, but bigger storage, slower search, higher cost. Common sizes: 384 $\rightarrow$ 768 $\rightarrow$ 1024 $\rightarrow$ 1536.
- **Bigger / newer models** usually retrieve better — compare them on the public **MTEB** leaderboard, not by feel.
- **Be consistent**: embed the query with the *same* model you used for the documents, or the vectors aren't comparable.
- **Chinese / Hindi**: the English models above won't work well. Use a multilingual embedder — `intfloat/multilingual-e5-base` (768) or `BAAI/bge-m3` (1024); lighter `paraphrase-multilingual-MiniLM-L12-v2` (384). Chinese-specific: `BAAI/bge-large-zh-v1.5` ; Hindi / Indic: `l3cube-pune/indic-sentence-similarity-sbert` .

Play later: set `EMBED_MODEL_NAME` above to `all-mpnet-base-v2` (768-dim) and re-run — watch the dimension change and whether retrieval improves. The next cell embeds one sentence with two models so you can see the sizes side by side.

```
In [104... # TRY LATER (downloads a second model, ~400 MB): compare two embedders of different size.
from sentence_transformers import SentenceTransformer
for name in ['all-MiniLM-L6-v2', 'all-mpnet-base-v2']:
    v = SentenceTransformer(name).encode(['cancel my credit card'])[0]
    print(f'{name:22s} -> {len(v):4d} dims    first 5: {v[:5].round(3)}')
```

```
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 13806.75it/s]
all-MiniLM-L6-v2      -> 384 dims    first 5: [ 0.015  0.047 -0.008  0.048 -0.028]
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 8591.61it/s]
all-mpnet-base-v2     -> 768 dims    first 5: [-0.028 -0.019 -0.006 -0.018  0.028]
```

Stage 2 — Retrieve

Embed the question the same way, then find the `TOP_K` chunks whose vectors are nearest. This is your 'vector store' in miniature — a real one (Pinecone, Milvus, pgvector) does this at massive scale.

```
In [105... import numpy as np

def cosine_top_k(query_vec, doc_vecs, k):
    # vectors are L2-normalised, so a dot product = cosine similarity
    sims = doc_vecs @ query_vec
```

```

    idx = np.argsort(-sims)[:k]
    return [(int(i), float(sims[i])) for i in idx]

# TRY THIS (knob): change how many chunks we retrieve.
TOP_K = 3

```

```

In [106... def retrieve(query):
    q = embed([query])[0]
    hits = cosine_top_k(q, doc_vecs, TOP_K)
    return [CHUNKS[i] for i, _ in hits]

# Demo: print the query, then the top-k chunks WITH their cosine similarity to it.
demo_q = 'what is RAG?'
print('QUERY:', demo_q)
print(f'Top-{TOP_K} chunks by cosine similarity:')
for i, sim in cosine_top_k(embed([demo_q])[0], doc_vecs, TOP_K):
    print(f'  sim={sim:.3f}  {CHUNKS[i]}')

```

QUERY: what is RAG?

Top-3 chunks by cosine similarity:

sim=0.494 the vectors nearest the question. Chunking and the number of retrieved chunks (top-k) usually decide RAG quality more than the model does.

sim=0.335 Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks most similar to a question, and generate an answer from those

sim=0.317 PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation models. Its DBS-GPT assistant pairs rented models with an internal knowledge base, which is RAG.

Stage 3 — Generate

Build a prompt from the retrieved chunks and let a small local **instruct** model answer **using only that context**. This is the one layer you'd often *rent* in production — here we run it for free.

```

In [107... from transformers import pipeline

# TRY THIS (knob): any small open instruct model. Qwen2.5-0.5B is a solid CPU default;
# 'HuggingFaceTB/SmolLM2-360M-Instruct' is lighter/faster; bigger models answer better.
GEN_MODEL_NAME = 'Qwen/Qwen2.5-0.5B-Instruct'
generator = pipeline('text-generation', model=GEN_MODEL_NAME) # downloads once; CPU-friendly

```

```
def answer(query):
    context = '\n'.join(retrieve(query))
    messages = [
        {'role': 'system', 'content': 'Answer using ONLY the context. If the answer is not in the context, say you'},
        {'role': 'user', 'content': f'Context:\n{context}\n\nQuestion: {query}'},
    ]
    out = generator(messages, max_new_tokens=200, do_sample=False)
    return out[0]['generated_text'][-1]['content']
```

Loading weights: 100%|██████████| 290/290 [00:00<00:00, 6800.33it/s]

Try it — see the whole pipeline

For one question, this shows the **retrieved chunks** (the exact context), a **peek at the embeddings** (meaning as numbers), and the **final answer**. Change `QUERY` and re-run.

Other small, open local models to try — set `GEN_MODEL_NAME` in **Stage 3**, re-run Stage 3, then run this cell:

- `Qwen/Qwen2.5-0.5B-Instruct` — default, good balance (~1 GB)
- `HuggingFaceTB/SmolLM2-360M-Instruct` — lighter / faster (~0.7 GB)
- `Qwen/Qwen2.5-1.5B-Instruct` — better answers, slower (~3 GB)
- `TinyLlama/TinyLlama-1.1B-Chat-v1.0` — a classic small chat model

For / हिन्दी: the default **Qwen2.5** already handles **Chinese** very well (Alibaba) and Hindi reasonably; SmolLM2 / TinyLlama are English-focused. Stronger options: `Qwen/Qwen2.5-1.5B-Instruct` (multilingual), `sarvamai/sarvam-1` (India-built, Hindi / Indic), `google/gemma-2-2b-it` (multilingual; needs a Hugging Face login).

```
In [108.. # TRY THIS (knob): ask anything answerable from the notes above.
# e.g. "What did DBS build versus buy?" · "Why do evals matter?" · "What costs the most?"
QUERY = "What is RAG, in one sentence?"

# --- 1) EMBED the question, then RETRIEVE the top-k nearest chunks (the context) ---
qvec = embed([QUERY])[0]
hits = cosine_top_k(qvec, doc_vecs, TOP_K)          # list of (chunk_index, similarity)

print("=" * 72)
print("QUESTION:", QUERY)
```

```

print("=" * 72)

print(f"\nTOP--{TOP_K} RETRIEVED CHUNKS (exactly the context the model is given):")
for rank, (i, sim) in enumerate(hits, 1):
    print(f" {rank}. sim={sim:.2f} {CHUNKS[i]}")

# --- 2) PEEK at the embeddings: 'meaning' is just a long list of numbers ---
print(f"\nEMBEDDING PEEK (each vector has {len(qvec)} dims; showing the first 8):")
print(" question : ", qvec[:8].round(3))
print(" best chunk :", doc_vecs[hits[0][0]][:8].round(3))

# --- 3) GENERATE: the model answers using ONLY those chunks ---
print("\nANSWER:")
print(" ", answer(QUERY))
print("=" * 72)

```

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

```

=====
QUESTION: What is RAG, in one sentence?
=====

```

TOP-3 RETRIEVED CHUNKS (exactly the context the model is given):

1. sim=0.46 the vectors nearest the question. Chunking and the number of retrieved chunks (top-k) usually decide RAG quality more than the model does.
2. sim=0.33 Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks most similar to a question, and generate an answer from those
3. sim=0.28 PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation models. Its DBS-GPT assistant pairs rented models with an internal knowledge base, which is RAG.

EMBEDDING PEEK (each vector has 384 dims; showing the first 8):

```

question : [-0.022  0.133  0.054  0.016 -0.094  0.029  0.085  0.035]
best chunk : [-0.042  0.002  0.052 -0.049 -0.007 -0.046 -0.049  0.032]

```

ANSWER:

RAG stands for Retrieval-Augmented Generation, a type of natural language processing technique that augments text retrieval by augmenting the database with additional information about the topics or contexts within the document.

How the model uses those chunks (the mechanics)

The `TOP_K` chunks aren't sent one at a time — they're concatenated into **one prompt** (instruction + all chunks + your question), and the model reads it in a single pass.

- **All chunks are in view.** A transformer uses **self-attention**, so while writing each word of the answer it can look at *every* token — all the retrieved chunks and your question. The answer is generated token by token, each conditioned on that whole context.
- **It doesn't average or vote.** The model leans on the chunk(s) that actually answer the question and mostly ignores the rest — and can *combine* facts across chunks when needed.
- **Irrelevant chunks aren't free.** They still cost tokens and can act as **distractors** — the noise side of the `TOP_K` knob (C3): too few misses the answer, too many dilutes it.
- **Position matters ("lost in the middle").** Models attend more to the start and end of the context than the middle, so a great chunk buried in the middle can be under-used — which is why **re-ranking** (best chunk first) helps (C3, *beyond naive RAG*).
- **"Use ONLY the context" is a strong bias, not a guarantee.** The model can still drift or invent — exactly why we measure **faithfulness / groundedness** in the evals section (C4).

Ties to the class: the model is the *rented* layer (C1); retrieval + chunking + top-k + re-ranking are the levers you *own* (C3); evals tell you whether the answer is actually grounded (C4).

Now experiment

You did not have to write RAG from scratch — you ran it. Now turn the knobs and watch the answer change:

1. **Change `TOP_K`** to 1, then 6 (retrieve cell), and **`CHUNK_WORDS`** to 15 (re-run from the chunking cell). These two knobs move retrieval more than the model does.
2. **Change `QUERY`** to something the notes *don't* cover (e.g. *"What is the capital of France?"*). Watch it either say it doesn't know, or make something up — that's a **silent failure**, and why we need evals.
3. **Break retrieval on purpose:** set `TOP_K = 0` or pass an unrelated query, and see the answer degrade even though nothing errors out.
4. **Compare the two notebooks:** run the same `QUERY` here and in the *other* RAG notebook. Same pipeline — only the embed and generate cells differ (local vs API). That difference *is* build-vs-buy at the model layer.

Evaluate it — the seed of A1

You turned the knobs by eye. Now **measure** it — AI fails silently. Two levels (from C4):

- **L1 — assertions:** deterministic true/false checks in code (is an out-of-scope question refused? does the answer contain the expected fact? is it short enough?).
- **L2 — a judge:** for what you can't assert, ask a model (or a human) to score. Here: **faithfulness** — is the answer supported by the retrieved chunks?

This tiny harness is the exact shape of **A1**. (On the small local model some checks may fail — that's the point: measure, don't assume.)

How the L1 checks work (and how to design your own)

L1 = **assertions** — each check is a plain true/false test in code. There are two kinds of cases:

- **In-scope** (`must_include`) — the answer *must contain* an expected fact (we check for a keyword). PASS = the fact is present.
- **Out-of-scope** (`must_not_include`) — the model *must refuse* ("I don't know...") **and must not leak** the real-world answer. PASS = refused **and** did not leak.

Why “refuse AND no leak”? A refusal-phrase check *alone* gives a **false PASS** on a hedging answer like “I cannot find it in the context, but the capital of France is Paris.” — it contains a refusal phrase yet still leaks *Paris*. Requiring `not leaked` catches that.

Why we print the answer. A PASS you can't see is a PASS you can't trust. Printing the model's answer under each question is the core habit — *read your data* — so you can confirm the check is right (and spot brittle ones).

Heuristics are brittle — both ways: a stray keyword can *false-PASS* a wrong answer; a valid refusal you didn't list (e.g. “the context doesn't mention that”) can *false-FAIL*. That brittleness is exactly why open-ended quality goes to **L2** (an LLM-as-judge) — keep L1 for the crisp, checkable things.

Designing yours (A1): for each case decide what a good answer must **contain** (`must_include`), must **avoid** (`must_not_include`), and any **format** bounds (length, valid JSON, no PII). Aim for ~10 cases in your own domain.

```

In [109... # A tiny labelled eval set: each case knows what a good answer looks like.
EVALSET = [
    {'q': 'What is RAG, in one sentence?', 'in_scope': True, 'must_include': ['retriev']},
    {'q': 'What did DBS build versus buy?', 'in_scope': True, 'must_include': ['ada']},
    {'q': 'What usually decides RAG quality?', 'in_scope': True, 'must_include': ['chunk']},
    {'q': 'What is the capital of France?', 'in_scope': False, 'must_not_include': ['paris']},
]

REFUSALS = ['do not know', "don't know", 'not in the context', 'no information', 'cannot find', 'not mentioned', 'n

def l1_checks(case, ans):
    a = ans.lower()
    checks = {}
    if case['in_scope']:
        checks['mentions expected fact'] = all(k in a for k in case.get('must_include', []))
    else:
        refused = any(r in a for r in REFUSALS)
        leaked = any(bad in a for bad in case.get('must_not_include', []))
        # out-of-scope PASSES only if it refuses AND does not leak the real-world answer
        checks['refuses & no leak'] = refused and not leaked
    checks['answer <= 400 chars'] = len(ans) <= 400
    return checks

passed = total = 0
for case in EVALSET:
    ans = answer(case['q'])
    print(f"\nQ: {case['q']}")
    print(f"A: {ans.strip()}") # print the answer so every PASS/FAIL is verifiable
    for name, ok in l1_checks(case, ans).items():
        total += 1; passed += int(ok)
        print((' PASS' if ok else ' FAIL'), '|', name)
print(f"\nL1 pass-rate: {passed}/{total} = {passed/total:.0%}")

```

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Q: What is RAG, in one sentence?

A: RAG stands for Retrieval-Augmented Generation, a type of natural language processing technique that augments text retrieval by augmenting the database with additional information about the topics or contexts within the document.

PASS | mentions expected fact

PASS | answer <= 400 chars

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Q: What did DBS build versus buy?

A: DBS built its own data platform (ADA) and a responsible-data framework called PURE

PASS | mentions expected fact

PASS | answer <= 400 chars

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Q: What usually decides RAG quality?

A: The vectors nearest to the question typically decide RAG quality more than the model does.

FAIL | mentions expected fact

PASS | answer <= 400 chars

Q: What is the capital of France?

A: France's capital is Paris.

FAIL | refuses & no leak

PASS | answer <= 400 chars

L1 pass-rate: 6/8 = 75%

```
In [110... # L2 (proxy): is the answer grounded in the retrieved chunks?
# A small local model can't reliably judge itself, so we use a lexical-overlap proxy.
# In the API notebook this becomes a real LLM-as-judge.
import re
def _content_words(t):
    return {w for w in re.findall(r'[a-z]{4,}', t.lower())}

for case in EVALSET:
    ctx = ' '.join(retrieve(case['q']))
    ans = answer(case['q'])
    aw, cw = _content_words(ans), _content_words(ctx)
    overlap = len(aw & cw) / (len(aw) + 1e-9)
    verdict = 'grounded' if overlap >= 0.5 else 'CHECK - low overlap'
```

```
print(f"{overlap:4.0%} {verdict:20s} {case['q'][:34]}")
print('\nThis L2 is a proxy; a real L2 uses an LLM-as-judge – see the API notebook.')
```

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

18% CHECK – low overlap What is RAG, in one sentence?

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

100% grounded What did DBS build versus buy?

[transformers] Both `max\_new\_tokens` (=200) and `max\_length` (=20) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

90% grounded What usually decides RAG quality?

0% CHECK – low overlap What is the capital of France?

This L2 is a proxy; a real L2 uses an LLM-as-judge – see the API notebook.

This is A1

A1 asks for exactly this, scaled up: **~10 cases** for a task in your industry, with **L1 assertions** (like above) **plus at least one L2 judge check**. Keep the harness runnable — the deliverable is the working checks, not an essay about evals.

What you just built

A complete **RAG pipeline, token-free**: `chunk -> embed -> store -> retrieve -> generate`, running entirely on this machine with open-source models. No API key, no bill.

The trade-off: a small local model gives simpler answers than a frontier API. That's the point — now open **PE6201_Class2_RAG_api** and run the *same* question to feel the difference in quality, latency, and cost.